MySocial: Componentes de Android e Kotlin usados no projeto

Guia de apoio para quem tem menos familiaridade com Android/Compose. Explica o que cada peça faz, por que ela foi usada aqui e como funciona por baixo dos panos, sempre com um trecho real do código do projeto.

Sumário

1. @Composable
2. remember e mutableStateOf
3. O operador "by" (property delegation)
4. LaunchedEffect
5. LazyColumn e items
6. rememberSaveable
7. TextField sem caixa (compositor estilo X)
8. ViewModel e AndroidViewModel
9. viewModelScope e coroutines
10. Dispatchers.IO e withContext
11. NavHost, NavController e rotas @Serializable
12. Scaffold e TopAppBar
13. AlertDialog (e como virou um menu de ações)
14. PrimaryTabRow e Tab
15. Image e painterResource
16. SQLiteOpenHelper
17. ContentValues e Cursor

1. Interface declarativa com Compose

`@Composable`

anotação do Jetpack Compose

É a anotação que transforma uma função Kotlin comum numa "peça de interface". Em vez de montar a tela imperativamente (criar um botão, depois adicionar ele na tela, depois configurar o clique), a gente só descreve "como a tela deveria ser" pro estado atual, e o Compose desenha isso sozinho.

Toda tela do projeto (`FeedScreen` , `LoginScreen` etc.) e todo componente reaproveitado (`CardPost` , `TopNavBar`) é uma função marcada com `@Composable` .

```
@Composable
fun CardPost(username: String, ...) {
    Card(...) { ... }
}
```

Sempre que algum dado que a função usa muda, o Compose chama a função de novo (isso se chama "recomposição") e só atualiza na tela o que realmente mudou.

2. Estado da tela

`remember` e `mutableStateOf`

androidx.compose.runtime

`mutableStateOf(valorInicial)` cria uma "caixinha" de estado: um valor que, quando muda, avisa o Compose pra redesenhar quem está usando ele. `remember { ... }` garante que esse valor não seja recriado do zero toda vez que a função for recomposta, ele "lembra" o valor entre recomposições.

```
var email by remember { mutableStateOf("") }
```

Usado em praticamente toda tela com campo de texto (`LoginScreen` , `RegisterScreen` , `PostScreen`) pra guardar o que o usuário está digitando.

> No projeto: campos de formulário, controle de "senha visível/escondida" no LoginScreen, e o id do post aguardando confirmação de exclusão no FeedScreen.

O operador "by" (property delegation)

recurso da linguagem Kotlin

Sem o `by`, pra usar o valor de dentro de um `mutableStateOf` seria preciso escrever `email.value` toda vez. Com `by`, o Kotlin "delega" a leitura e escrita da variável direto pro `mutableStateOf`, então `email` se comporta como uma String normal.

```
var loading by mutableStateOf(false)
    private set
```

Esse `private set` aparece em quase todo ViewModel do projeto: qualquer código pode ler o valor, mas só o próprio ViewModel pode alterá-lo. É assim que a tela nunca consegue mudar o estado "por fora", só através de uma função do ViewModel.

LaunchedEffect

androidx.compose.runtime

Roda um bloco de código suspenso (uma coroutine) quando a tela aparece, e roda de novo toda vez que a "chave" passada pra ele mudar. É a forma correta, dentro do Compose, de reagir a mudanças de estado ou disparar uma ação uma única vez.

```
LaunchedEffect(Unit) {
    postViewModel.list()
}
```

Com a chave `Unit` (que nunca muda), o bloco roda uma única vez, quando a tela é exibida pela primeira vez. É assim que o `FeedScreen` busca os posts assim que abre.

```
LaunchedEffect(authViewModel.done) {
    if (authViewModel.done) {
        authViewModel.consumeResult()
        onLoginSuccess()
    }
}
```

Já aqui, a chave é o próprio estado `done`. Toda vez que ele mudar (de `false` pra `true`, quando o login dá certo), o bloco roda de novo e navega pro feed. É o mesmo mecanismo usado pra exigir login: em `AppNavigation.kt`, um `LaunchedEffect(destination, currentUser)` redireciona pro Login sempre que alguém tenta abrir uma rota protegida sem estar logado.

3. Listas

LazyColumn e items

androidx.compose.foundation.lazy

É a versão "preguiçosa" (lazy) de uma coluna com rolagem. A diferença pra uma `Column` normal dentro de um `scroll` é que a `LazyColumn` só desenha na memória os itens que estão realmente visíveis na tela (mais uma pequena margem), em vez de desenhar os 500 posts do banco de uma vez. Isso é essencial pra performance de uma lista que pode crescer bastante, como o feed.

```
LazyColumn {
    item { CircularProgressIndicator() }
    items(postViewModel.posts, key = { it.id }) { p
        CardPost(...)
    }
}
```

`item { }` adiciona um elemento único (tipo um cabeçalho). `items(lista, key = ...)` adiciona um elemento pra cada posição da lista. O parâmetro `key` ajuda o Compose a saber "qual item é qual" mesmo que a lista seja reordenada, evitando redesenhar tudo à toa.

> Usado no Feed, no Perfil (lista de posts do usuário) e na tela de Seguidores/Seguindo.

rememberSaveable

androidx.compose.runtime.saveable

Igual ao `remember`, mas sobrevive também a uma troca de configuração do sistema (como girar a tela), porque o valor é salvo num `Bundle`, não só em memória. Usado pra guardar qual post está aguardando confirmação de exclusão, pra não perder esse estado se o usuário girar o celular no meio da confirmação.

```
var deleteId by rememberSaveable { mutableStateOf<L
```

TextField sem caixa (compositor estilo X)

androidx.compose.material3 (mesma família do OutlinedTextField do professor)

`TextField` é o "primo" do `OutlinedTextField` que o professor ensina: os dois vêm do mesmo pacote e funcionam igual (`value`, `onValueChange`, etc.), só muda o visual padrão. O compositor de post/comentário usa `TextField` com as cores do fundo e da linha inferior deixadas transparentes, pra não aparecer nenhuma caixa, só o texto digitado, igual ao campo "O que está acontecendo?" do X.

```
TextField(
    value = content,
    onValueChange = { ... },
    placeholder = { Text("O que está acontecendo?")
    minLines = 6,
    colors = TextFieldDefaults.colors(
        focusedContainerColor = Color.Transparent,
        unfocusedContainerColor = Color.Transparent
        focusedIndicatorColor = Color.Transparent,
        unfocusedIndicatorColor = Color.Transparent
    )
)
```

`minLines` faz o campo já nascer com várias linhas de altura, em vez de uma linha só que cresce aos poucos. O resto (limite de caracteres, validação) continua exatamente igual ao `OutlinedTextField` usado nos outros formulários do projeto (Login, Cadastro, Editar Perfil).

4. ViewModel e coroutines

ViewModel e AndroidViewModel

androidx.lifecycle

Um `ViewModel` é uma classe que guarda o estado de uma tela de um jeito que sobrevive a recomposições e a mudanças de configuração (como girar o celular), porque ele não é recriado junto com a tela, ele fica "amarrado" ao ciclo de vida da tela/rota, não ao desenho dela.

`AndroidViewModel` é um `ViewModel` que já recebe o `Application` (contexto da aplicação) no construtor, necessário aqui porque os DAOs (`UserDAO`, `PostDAO`...) precisam de um `Context` pra abrir o banco.

```
class PostViewModel(application: Application)
    : AndroidViewModel(application) {
    private val dao = PostDAO(application)
    var posts by mutableStateOf<List<Post>>(emptyLi
        private set
    ...
}
```

A tela pega uma instância dele assim: `postViewModel: PostViewModel = viewModel()`. O Compose cuida de criar e reaproveitar essa instância sozinho.

viewModelScope e coroutines (launch)

androidx.lifecycle / kotlinx.coroutines

Uma coroutine é uma forma de rodar código que pode "pausar" (por exemplo, esperando o banco responder) sem travar a interface do usuário enquanto espera. `viewModelScope` é um escopo de coroutine que já é cancelado automaticamente se o ViewModel for destruído, evitando vazamento de tarefa rodando à toa.

```
viewModelScope.launch {
    try {
        done = withContext(Dispatchers.IO) { op() }
    } catch (e: Exception) {
        message = "Ocorreu um erro inesperado."
    } finally {
        loading = false
    }
}
```

Esse padrão (função `run` privada dentro de cada ViewModel) se repete em `AuthViewModel`, `PostViewModel`, `ProfileViewModel` e `FollowsViewModel`: controla o `loading`, roda a operação, e transforma qualquer erro numa mensagem amigável pra tela mostrar.

Dispatchers.IO e withContext

kotlinx.coroutines

Acessar o banco de dados é uma operação de entrada/saída (I/O) que pode demorar uns milissegundos. Se isso rodasse na thread principal (a que desenha a tela), o app congelaria por um instante. `Dispatchers.IO` é um grupo de threads feito pra esse tipo de tarefa.

```
withContext(Dispatchers.IO) {
    dao.create(userId, trimmedContent)
}
```

`withContext` troca pra essa thread só durante o bloco, executa o código ali, e volta pra thread principal automaticamente assim que termina, já com o resultado pronto pra atualizar o estado da tela com segurança.

5. Navegação entre telas

NavHost, NavController e rotas @Serializable

androidx.navigation.compose

`NavController` é o objeto que sabe pra onde o app está navegando e guarda o histórico de telas (pra o botão "voltar" funcionar). `NavHost` é o componente que declara todas as rotas possíveis e qual tela corresponde a cada uma.

Cada rota é uma classe ou objeto Kotlin marcado com `@Serializable`, em vez de uma simples String como em projetos mais antigos. Isso dá segurança de tipos: é impossível, por exemplo, navegar pra tela de detalhe do post esquecendo de informar o `postId`, porque isso nem compila.

```
@Serializable
data class DetailPost(val postId: Long)

// ...
composable<DetailPost> { backStackEntry ->
    val rota = backStackEntry.toRoute<DetailPost>()
    DetailPostScreen(postId = rota.postId, ...)
}
```

Pra navegar, só se cria uma instância da rota: `navController.navigate(DetailPost(post.id)`

Scaffold e TopAppBar

androidx.compose.material3

`Scaffold` é um "esqueleto" de tela padrão do Material Design: já reserva o espaço certo pra uma barra do topo, conteúdo e eventualmente uma barra inferior, cuidando do espaçamento (`innerPadding`) sozinho. No projeto existe um único `Scaffold`, lá no `MainActivity`, que envolve a navegação inteira.

```
Scaffold(
    topBar = { if (showTopBar) TopNavBar(navControl
) { innerPadding ->
    AppNavigation(modifier = Modifier.padding(inner
}
```

`TopAppBar` (usado dentro do `TopNavBar`) é a barra fixa no topo com o nome do app e os ícones de ação (novo post, feed, perfil, sair). Ela só aparece quando a rota atual não é Login nem Cadastro.

AlertDialog (e como virou um menu de ações)

androidx.compose.material3

Uma janela que flutua por cima do resto da tela. O uso mais comum é pra confirmar uma ação irreversível, como excluir um post: em vez de excluir no primeiro clique, o botão "Excluir" só marca qual item deveria ser excluído, e o `AlertDialog` pede confirmação antes de chamar o ViewModel de verdade.

```
AlertDialog(
    onDismissRequest = { deleteId = null },
    title = { Text("Excluir post?") },
    confirmButton = {
        TextButton(onClick = {
            deleteId = null
            postViewModel.delete(post.id)
        }) { Text("Excluir") }
    },
    dismissButton = {
        TextButton(onClick = { deleteId = null }) {
    }
)
```

No `CardPost` e no `DetailPostScreen`, o mesmo componente é reaproveitado de um jeito diferente: em vez de "confirmar ou cancelar", o `text` do diálogo vira uma lista de botões ("Editar", "Excluir"), funcionando como um pequeno menu de ações. Ele só abre quando a pessoa clica no ícone de três pontinhos (`Icons.Default.MoreVert`) no cabeçalho do post, e só aparece pra quem é dono dele. É o mesmo componente do professor, só usado de um jeito a mais, pra esconder as opções de editar/excluir em vez de deixá-las sempre visíveis (igual ao " ⋮ " do X).

`PrimaryTabRow` e `Tab`

androidx.compose.material3 (não é do material do professor, mas é só um menu de abas)

É o menu de abas que troca o conteúdo da tela sem precisar navegar pra outra rota. Usado na tela de Seguidores/Seguindo: um `PrimaryTabRow` com dois `Tab` ("Seguidores" e "Seguindo"), e a aba marcada como `selected = true` decide qual lista é carregada.

```
var abaSelecionada by remember(userId) { mutableSta

PrimaryTabRow(selectedTabIndex = if (abaSelecionada
    Tab(
        selected = abaSelecionada == FOLLOWERS,
        onClick = { abaSelecionada = FOLLOWERS },
        text = { Text("Seguidores") }
    )
    Tab(
        selected = abaSelecionada == FOLLOW,
        onClick = { abaSelecionada = FOLLOW },
        text = { Text("Seguindo") }
    )
}
```

Trocar de aba só muda o valor de `abaSelecionada` (um estado comum, com `remember`), e um `LaunchedEffect(userId, abaSelecionada)` recarrega a lista certa sempre que esse valor muda, sem precisar voltar pra tela de perfil e clicar de novo.

`Image` e `painterResource`

androidx.compose.foundation / androidx.compose.ui.res (mesmo jeito que o professor usa imagem)

É como o Compose desenha uma imagem vinda de um recurso do projeto (pasta `res/drawable/`). `painterResource(id = R.drawable.algumacoisa)` carrega o arquivo, e `Image(painter = ..., contentDescription = ...)` desenha ele na tela. É exatamente o padrão que o professor usa no material dele.

```
Image(
    painter = painterResource(id = R.drawable.ic_pr
    contentDescription = "Foto de perfil",
    modifier = Modifier.size(Dimens.avatarSize)
)
```

O projeto usa um ícone de pessoa (`ic_profile_placeholder.xml`) como foto de perfil provisória, dentro de um círculo colorido (a cor de capa de cada usuário). É um `vector drawable`, ou seja, um desenho vetorial (o equivalente, dentro do Android, a um SVG), guardado localmente no projeto, sem depender de nenhum arquivo baixado de fora. No futuro, trocar esse ícone por uma foto de verdade enviada pelo usuário é só trocar o `painterResource` por um `painter` que carregue a foto, sem mudar o resto da tela.

6. Banco de dados

SQLiteOpenHelper

android.database.sqlite

É a classe base do Android pra criar e manter um banco SQLite, sem precisar de nenhuma biblioteca externa (nada de Room). Se estende essa classe e implementa dois métodos: `onCreate` (roda só na primeira vez que o app abre, cria todas as tabelas do zero) e `onUpgrade` (roda quando o número de versão do banco aumenta, aplica as mudanças de schema nos dados que já existem).

```
class AppDatabase(context: Context) : SQLiteOpenHelp
    context.applicationContext,
    File(context.noBackupFilesDir, "mysocial.db").ab
    null,
    2 // versão do schema
) {
    override fun onCreate(db: SQLiteDatabase) { ...
    override fun onUpgrade(db, oldVersion, newVersio
}
```

`writableDatabase` abre o banco em modo leitura e escrita (usado pra INSERT/UPDATE/DELETE). `readableDatabase` abre só pra leitura (usado nas buscas). Os DAOs sempre usam `AppDatabase(context).use { db -> ... }`, o `use` garante que a conexão é fechada sozinha ao final, mesmo se der erro no meio.

ContentValues e Cursor

android.content / android.database

`ContentValues` é um mapa de "nome da coluna" para "valor", usado pra montar um INSERT ou UPDATE sem concatenar texto (o que abriria brecha pra SQL Injection).

```
val values = ContentValues().apply {
    put("username", username)
    put("email", email)
}
db.writableDatabase.insertOrThrow("users", null, va
```

`Cursor` é o objeto que representa o resultado de uma consulta (`query` ou `rawQuery`), uma linha por vez. `cursor.moveToNext()` avança pra próxima linha, e `cursor.getLong(0)` / `getString(1)` leem o valor de cada coluna pela posição.

```
db.readableDatabase.query(
    "users", arrayOf("id", "username", "email"),
    "id = ?", arrayOf(id.toString()),
    null, null, null
).use { cursor ->
    if (cursor.moveToFirst())
        User(cursor.getLong(0), cursor.getString(1)
    else null
}
```

Repare no `"id = ?"` com `arrayOf(id.toString())` logo depois: esse é o "bind parameter". O SQLite substitui o `?` pelo valor de forma segura, nunca interpretando o conteúdo como parte do comando SQL. É assim que todo o projeto se protege de SQL Injection.

Documento gerado como material de apoio interno do projeto MySocial (trabalho acadêmico), pra ajudar a equipe a entender e defender o código nas partes que não desenvolveu diretamente.